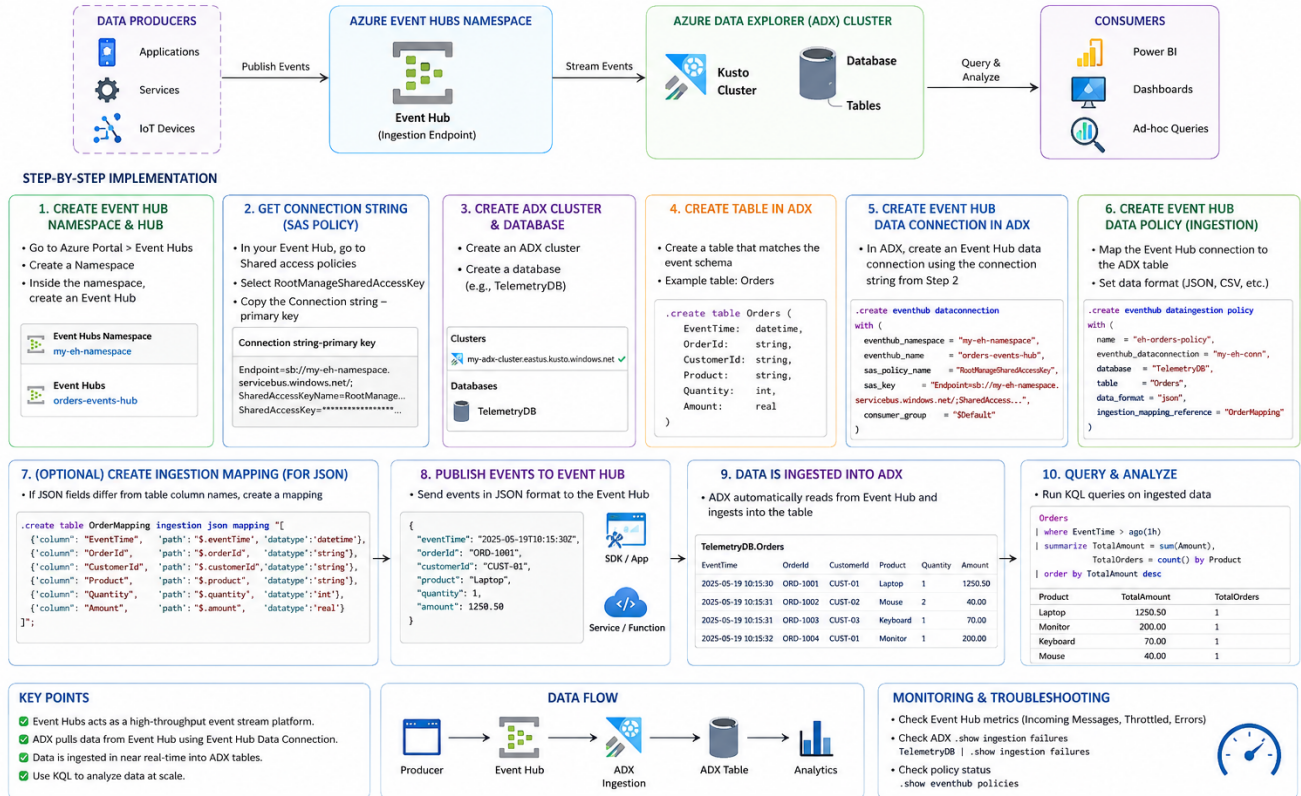


END-TO-END: CONNECT AZURE DATA EXPLORER (ADX) WITH AZURE EVENT HUBS AND PUBLISH EVENT HUB DATA INTO ADX

END TO END: CONNECT AZURE DATA EXPLORER (ADX) WITH AZURE EVENT HUBS AND PUBLISH EVENT HUB DATA INTO ADX



Generated architecture image included above.

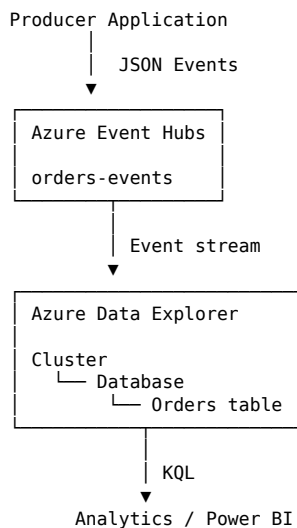
Absolutely. Think of the integration as a streaming pipeline:

Application → Azure Event Hub → Azure Data Explorer (ADX) → KQL / Power BI

End-to-end architecture

The important point is that your application does not directly insert data into ADX.

Instead:



The generated diagram above shows the complete flow.

1. What are we actually building?

Let's use a realistic example.

Imagine an e-commerce application generating order events:

```
{
  "eventTime": "2026-08-09T21:30:15Z",
  "orderId": "ORD-1001",
  "customerId": "CUST-101",
  "product": "Laptop",
  "quantity": 1,
  "amount": 75000
}
```

The application publishes this event to:

```
Azure Event Hub
┆
┆ orders-events
```

ADX continuously consumes those events and stores them in an ADX table:

```
Orders
-----
eventTime
orderId
customerId
product
quantity
amount
```

Then you can run:

```
Orders
| summarize TotalSales = sum(amount)
```

2. Step 1 — Create Event Hubs

In Azure Portal:

```
Azure Portal
↓
Event Hubs
↓
Create Event Hubs Namespace
↓
Create Event Hub
```

For example:

```
Namespace:
my-company-events

Event Hub:
orders-events
```

Think of an Event Hub as a high-speed mailbox for events.

Applications continuously put messages into it.

```
Application 1 ┌─┐
Application 2 ┤─┤ → orders-events
Application 3 └─┘
```

3. Step 2 — Application publishes events

Your Python application could publish events using the Azure Event Hubs SDK.

Conceptually:

```
event = {
    "eventTime": "2026-08-09T21:30:15Z",
    "orderId": "ORD-1001",
    "customerId": "CUST-101",
    "product": "Laptop",
    "quantity": 1,
    "amount": 75000
}

event_hub.send(event)
```

So the Event Hub contains events like:

```
Event 1 → ORD-1001
Event 2 → ORD-1002
Event 3 → ORD-1003
Event 4 → ORD-1004
...
```

4. Step 3 — Create Azure Data Explorer

Now create an ADX cluster.

For example:

```
ADX Cluster
|
```

└─ Database: SalesDB

Inside the database we'll create:

SalesDB
└─ Orders

ADX is essentially a large-scale analytics engine optimized for time-series, logs, telemetry and event data.

The query language is KQL — Kusto Query Language.

5. Step 4 — Create the ADX table

We need somewhere to store the incoming Event Hub data.

In ADX:

```
.create table Orders (  
    eventTime: datetime,  
    orderId: string,  
    customerId: string,  
    product: string,  
    quantity: int,  
    amount: real  
)
```

Now ADX knows the expected schema.

Think of this as:

Orders

eventTime	orderId	customerId	product	quantity	amount
21:30:15	ORD-1001	CUST-101	Laptop	1	75000
21:30:20	ORD-1002	CUST-102	Mouse	2	2000

6. The most important step — connect Event Hub to ADX

This is where the actual integration happens.

In ADX, you create an:

Event Hub Data Connection

Conceptually:

Event Hub
└─ Connection
▼
ADX Data Connection
└─
Orders table

The connection tells ADX:

"Go to this Event Hub, continuously read its events, and ingest them into this ADX database/table."

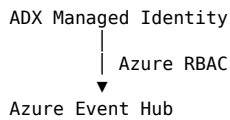
7. How does ADX authenticate to Event Hub?

ADX needs permission to read the Event Hub.

There are two common approaches:

Option A — Managed Identity

Preferred in production.



You give the ADX identity appropriate Event Hubs permissions, such as receiving events.

Option B — SAS policy

You can provide an Event Hub SAS connection string/policy.

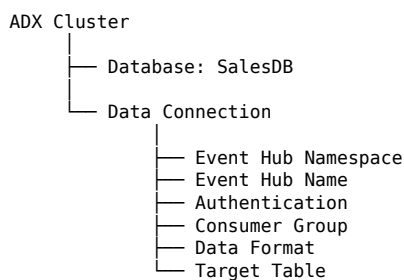
For example:

```
Endpoint=sb://my-company-events.servicebus.windows.net/;  
SharedAccessKeyName=RootManageSharedAccessKey;  
SharedAccessKey=xxxxxxxx
```

For enterprise production environments, Managed Identity + Azure RBAC is generally preferable because you don't have to manage secrets in connection strings.

8. Create the Event Hub Data Connection

Conceptually, the configuration looks like:



For example:

Event Hub Namespace:
my-company-events

Event Hub:
orders-events

Consumer Group:
adx-consumer

Database:
SalesDB

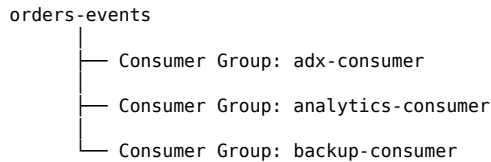
Table:
Orders

Data Format:
JSON

9. Why do we need a Consumer Group?

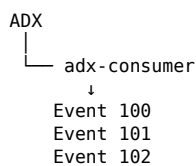
This is an important Event Hubs concept.

Imagine:

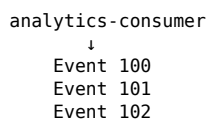


Each consumer group maintains its own reading position.

For example:



Another application can independently read:



This allows multiple consumers to process the same event stream independently.

10. But how does JSON become ADX columns?

This is where ingestion mapping becomes important.

Suppose Event Hub sends:

```
{
  "eventTime": "2026-08-09T21:30:15Z",
  "orderId": "ORD-1001",
  "customerId": "CUST-101",
  "product": "Laptop",
  "quantity": 1,
  "amount": 75000
}
```

ADX table:

```
eventTime
orderId
customerId
product
quantity
amount
```

If names match, the mapping can be straightforward.

But imagine your JSON is:

```
{
```

```

    "timestamp": "...",
    "order_number": "ORD-1001",
    "customer": "CUST-101",
    "item": "Laptop"
}

```

while ADX expects:

```

eventTime
orderId
customerId
product

```

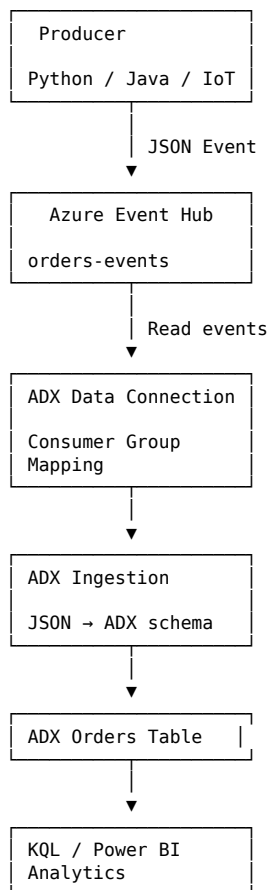
Then we need an ingestion mapping:

JSON field		ADX column
timestamp	——>	eventTime
order_number	——>	orderId
customer	——>	customerId
item	——>	product

This is one of the key concepts you should understand for Azure Data Engineering interviews.

11. Complete ingestion flow

Now put everything together:



12. What happens when a real event arrives?

Suppose your application generates:

```
{
  "eventTime": "2026-08-09T21:35:10Z",
  "orderId": "ORD-2001",
  "customerId": "CUST-500",
  "product": "Monitor",
  "quantity": 2,
  "amount": 30000
}
```

Step 1

Application sends it:

```
Application
  ↓
Event Hub
```

Step 2

Event Hub stores the event temporarily as part of the event stream.

```
orders-events
├── Event 1
├── Event 2
├── Event 3
└── Event 4 ← new event
```

Step 3

ADX's Event Hub data connection reads it.

```
Event Hub
  ↓
ADX Data Connection
```

Step 4

ADX processes the JSON.

```
JSON
  ↓
Mapping
  ↓
ADX schema
```

Step 5

ADX ingests it.

```
Orders table
```

eventTime	orderId	product	amount
2026-08-09 21:35:10	ORD-2001	Monitor	30000

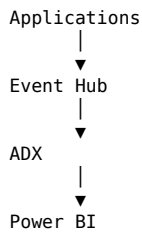
Step 6

You query it with KQL:

```
Orders
| where product == "Monitor"
| project eventTime, orderId, quantity, amount
```

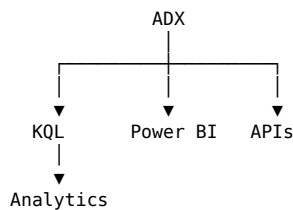

13. Where does Power BI come into the picture?

Now the architecture becomes:



Power BI can query ADX for visualization.

For example:



You could build a dashboard showing:

SALES DASHBOARD		
Total Sales	₹12.5 Crore	
Orders	45,230	
Laptop		₹5.2 Cr
Monitor		₹3.1 Cr
Mouse		₹1.2 Cr

14. Event Hub vs ADX — very important

Don't think that Event Hub and ADX do the same thing.

| Component | Main responsibility |

---|---

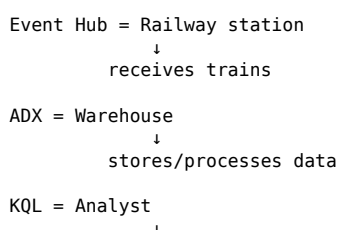
| Event Hub | Receive and buffer high-volume events |

| ADX | Store, query and analyze those events |

| KQL | Query/analyze data in ADX |

| Power BI | Visualize the results |

A simple analogy:



asks questions
Power BI = Dashboard
↓
shows results

15. What happens if ADX temporarily cannot ingest?

This is another important real-world scenario.

Suppose:

Application
↓
Event Hub
↓
X
ADX temporarily unavailable

The events remain in Event Hubs according to the configured retention period.

Later:

ADX becomes available
↓
ADX continues consuming
↓
previous events are processed

This is one reason Event Hubs is useful as a buffer between producers and downstream consumers.

16. What should you monitor?

In production, you don't just create the connection and forget about it.

Monitor:

Event Hub

Incoming Messages
Incoming Bytes
Outgoing Messages
Throttled Requests
Consumer Lag
Errors

ADX

Monitor:

Ingestion failures
Ingestion latency
Data connection health
Query performance
Cluster resource utilization

ADX also provides KQL-based diagnostics for ingestion failures.

17. The interview answer

If an interviewer asks:

"How do you integrate Azure Event Hub with Azure Data Explorer?"

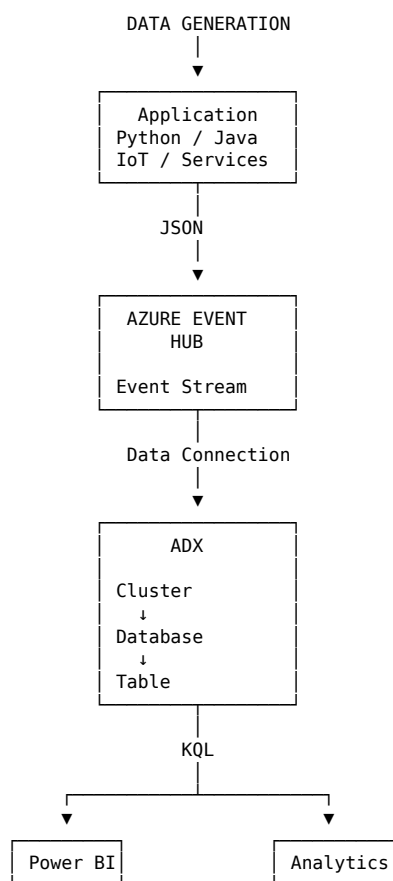
A strong answer would be:

"I create an Event Hub namespace and event hub for the incoming event stream. I create an ADX cluster and database with the target table. Then I configure an Event Hub data connection in ADX, specifying the Event Hub, consumer group, authentication mechanism, target database/table, and data format. If the incoming JSON schema differs from the ADX table schema, I configure an ingestion mapping. ADX then continuously consumes events from Event Hubs and ingests them into the target table. Once ingested, I use KQL for real-time analytics and can connect Power BI for visualization. In production, I prefer Managed Identity with Azure RBAC and monitor Event Hub metrics, consumer lag, and ADX ingestion failures."

That is a very good Azure Data Engineering interview answer.

18. The entire thing in one picture

The mental model I recommend you remember is:



The 7 things you should remember

Event Hub → Data Connection → Consumer Group → Mapping → ADX Table → KQL → Power BI

If you understand those seven pieces, you understand the core Event Hub → ADX streaming ingestion architecture.

If you're preparing for your Azure Data Engineering interviews, the next useful step is to understand exactly how to configure this in Azure Portal, including the screens, permissions, Managed Identity/RBAC, ingestion mapping, and a Python producer.